

Sri Lanka Institute of Information Technology



IT3012 : Intelligent Agents – Lab 5

Y3.S1.WD.AI.01.01

Malabe Campus

Student ID -	IT24100140
Student Name -	Prathiban HTP
Module Name -	Intelligent Agents
Module Number -	IT3012
LabSheet Number -	5
LabSheet Date -	2026/Sep/18

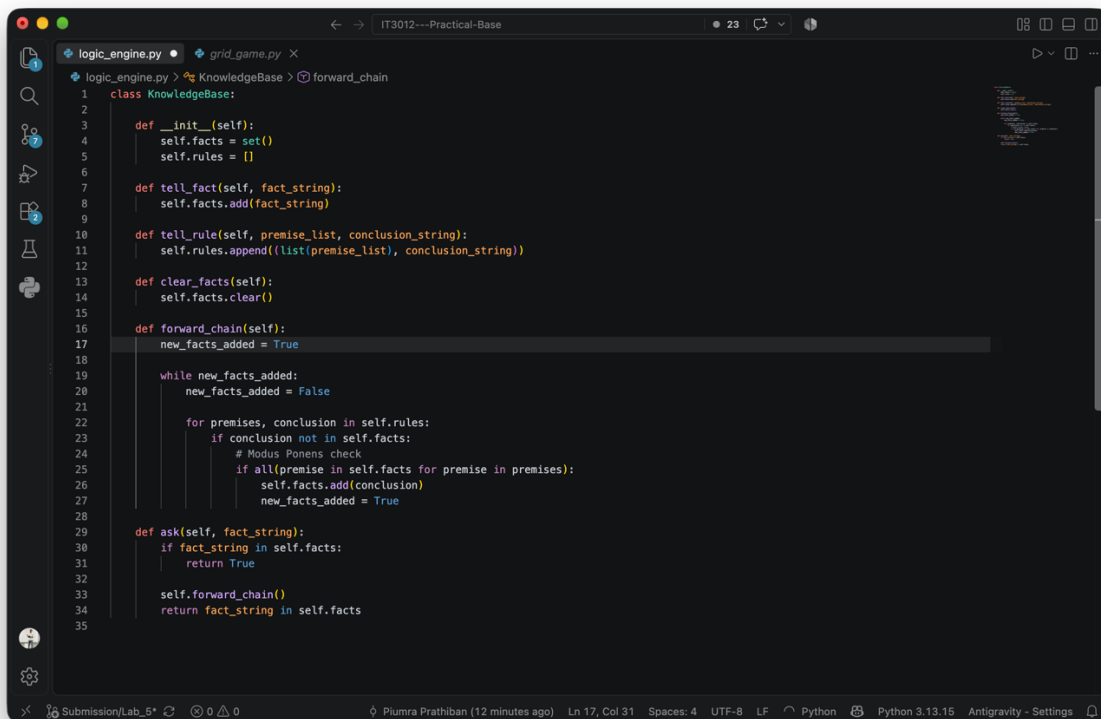
Step 1

Step 1.1

Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py`.
2. Implement a `KnowledgeBase` class based on the following structural requirements:

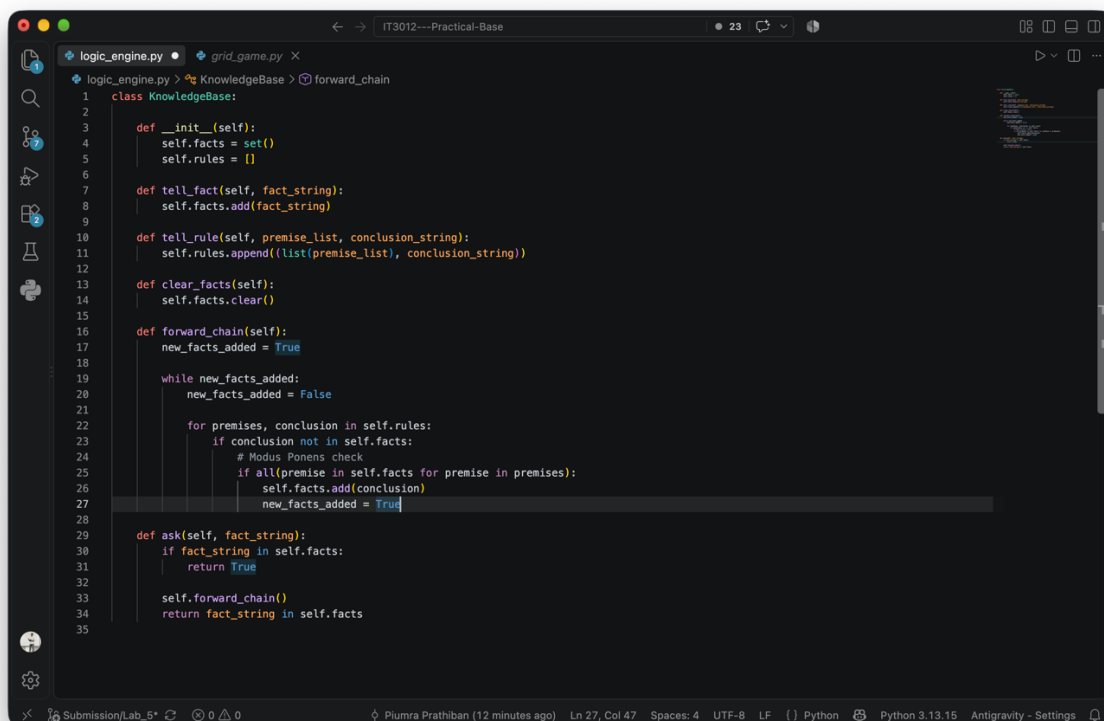


```
1 class KnowledgeBase:
2
3     def __init__(self):
4         self.facts = set()
5         self.rules = []
6
7     def tell_fact(self, fact_string):
8         self.facts.add(fact_string)
9
10    def tell_rule(self, premise_list, conclusion_string):
11        self.rules.append((list(premise_list), conclusion_string))
12
13    def clear_facts(self):
14        self.facts.clear()
15
16    def forward_chain(self):
17        new_facts_added = True
18
19        while new_facts_added:
20            new_facts_added = False
21
22            for premises, conclusion in self.rules:
23                if conclusion not in self.facts:
24                    # Modus Ponens check
25                    if all(premise in self.facts for premise in premises):
26                        self.facts.add(conclusion)
27                        new_facts_added = True
28
29    def ask(self, fact_string):
30        if fact_string in self.facts:
31            return True
32
33        self.forward_chain()
34        return fact_string in self.facts
35
```

Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

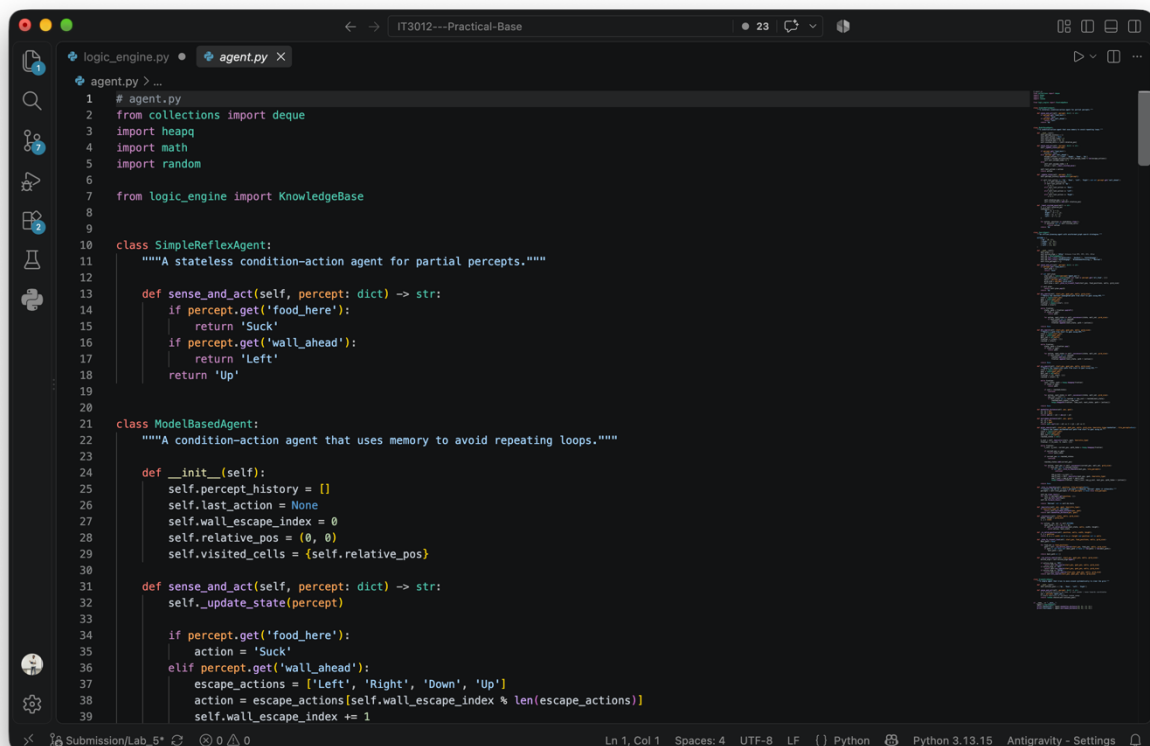


```
1 class KnowledgeBase:
2
3     def __init__(self):
4         self.facts = set()
5         self.rules = []
6
7     def tell_fact(self, fact_string):
8         self.facts.add(fact_string)
9
10    def tell_rule(self, premise_list, conclusion_string):
11        self.rules.append((list(premise_list), conclusion_string))
12
13    def clear_facts(self):
14        self.facts.clear()
15
16    def forward_chain(self):
17        new_facts_added = True
18
19        while new_facts_added:
20            new_facts_added = False
21
22            for premises, conclusion in self.rules:
23                if conclusion not in self.facts:
24                    # Modus Ponens check
25                    if all(premise in self.facts for premise in premises):
26                        self.facts.add(conclusion)
27                        new_facts_added = True
28
29    def ask(self, fact_string):
30        if fact_string in self.facts:
31            return True
32
33        self.forward_chain()
34        return fact_string in self.facts
35
```

Step 3.1: Defining the Game Constraints

Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In your main `agent.py`, instantiate the KB in the `__init__` method:
`self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
Rule 1: $\text{TargetVisible} \wedge \text{HasDust} \Rightarrow \text{SafeToEngage}$
Rule 2: $\text{SafeToEngage} \wedge \text{BloodseekerMissing} \Rightarrow \text{Retreat}$



```
1 # agent.py
2 from collections import deque
3 import heapq
4 import math
5 import random
6
7 from logic_engine import KnowledgeBase
8
9
10 class SimpleReflexAgent:
11     """A stateless condition-action agent for partial percepts."""
12
13     def sense_and_act(self, percept: dict) -> str:
14         if percept.get('food_here'):
15             return 'Suck'
16         if percept.get('wall_ahead'):
17             return 'Left'
18         return 'Up'
19
20
21 class ModelBasedAgent:
22     """A condition-action agent that uses memory to avoid repeating loops."""
23
24     def __init__(self):
25         self.percept_history = []
26         self.last_action = None
27         self.wall_escape_index = 0
28         self.relative_pos = (0, 0)
29         self.visited_cells = {self.relative_pos}
30
31     def sense_and_act(self, percept: dict) -> str:
32         self._update_state(percept)
33
34         if percept.get('food_here'):
35             action = 'Suck'
36         elif percept.get('wall_ahead'):
37             escape_actions = ['Left', 'Right', 'Down', 'Up']
38             action = escape_actions[self.wall_escape_index % len(escape_actions)]
39             self.wall_escape_index += 1
```

Step 3.2: Validating Feasibility in A*

Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

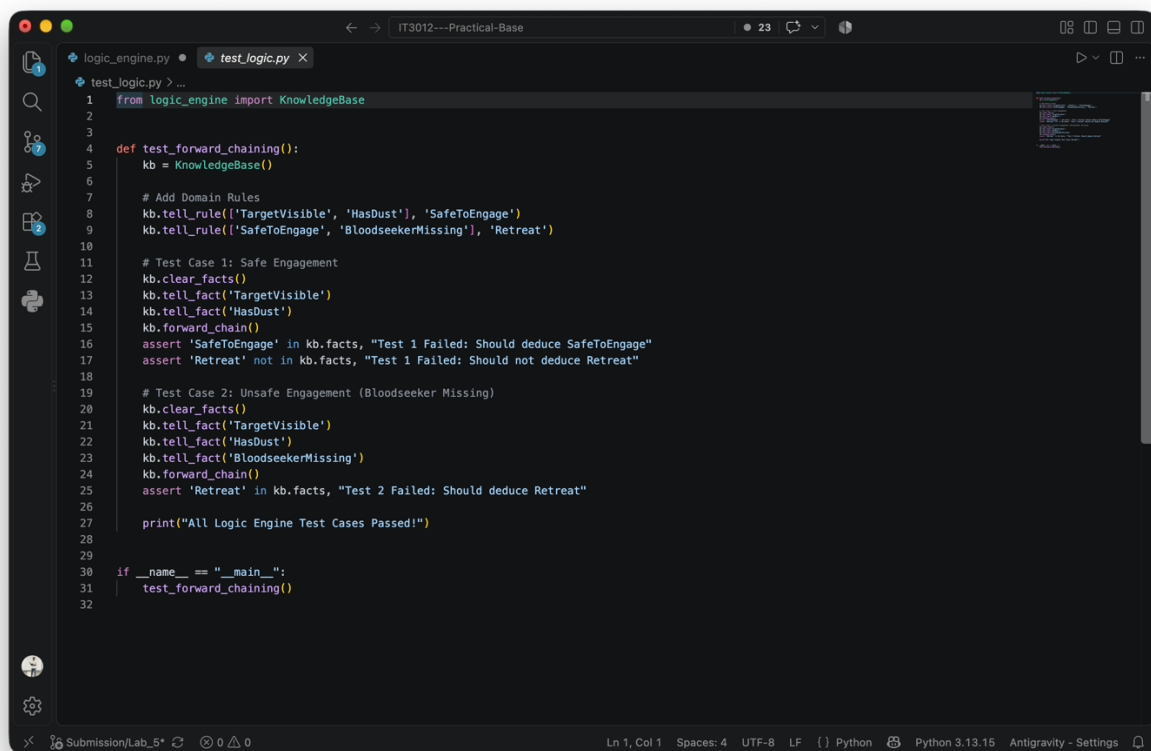
1. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
2. Before adding a neighbor to the `open_list`, clear the KB facts.
3. Feed the current percepts for that specific tile into the KB using `tell_fact()`.
4. Run `self.kb.forward_chain()`.
5. If 'Retreat' is deduced and exists in `self.kb.facts`, mark the tile as

seweb.sliit.lk/pluginfile.php/595879/mod_resource/content/0/Practical%2005%20-Student%20-%20Draft.html?embed=1

Infeasible. Skip it (do not add it to the open list), even if it is physically reachable.

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.



```
1 from logic_engine import KnowledgeBase
2
3
4 def test_forward_chaining():
5     kb = KnowledgeBase()
6
7     # Add Domain Rules
8     kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
9     kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')
10
11     # Test Case 1: Safe Engagement
12     kb.clear_facts()
13     kb.tell_fact('TargetVisible')
14     kb.tell_fact('HasDust')
15     kb.forward_chain()
16     assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce SafeToEngage"
17     assert 'Retreat' not in kb.facts, "Test 1 Failed: Should not deduce Retreat"
18
19     # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
20     kb.clear_facts()
21     kb.tell_fact('TargetVisible')
22     kb.tell_fact('HasDust')
23     kb.tell_fact('BloodseekerMissing')
24     kb.forward_chain()
25     assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat"
26
27     print("All Logic Engine Test Cases Passed!")
28
29
30 if __name__ == "__main__":
31     test_forward_chaining()
32
```

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop.

How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"?

Explain how the Knowledge Base enforces feasibility.

Reachability is **physical**: `_successors()` only generates positions that are inside the grid and not walls.

Feasibility is **logical**: `_tile_is_feasible()` checks a reachable tile using the Knowledge Base (KB). The KB adds the tile's percepts, runs `forward_chain()`, and rejects the tile if **Retreat** is derived.

Therefore, a tile can be **reachable but not feasible**. A* checks both before adding the node.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code?

What happens to the agent's code complexity if we add 50 new game rules?

The Knowledge Base keeps **rules and facts separate from the agent's movement code**. Instead of adding an ifstatement for every rule, `forward_chain()` applies all rules generically.

This makes the system easier to extend: adding 50 new rules only requires adding the rules to the KB, without making the movement code more complex. The inference engine remains the same.

3. Modus Ponens & Horn Clauses (Understand)

eb.sliit.lk/pluginfile.php/595879/mod_resource/content/0/Practical%2005%20-Student%20-%20Draft.html?embed=1

al 04: Informed Search

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` while loop acts as a mathematical execution of the *Modus Ponens* inference rule.

The rule:

`['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'`

is a **Horn clause**. Modus Ponens states that if all premises are true and they imply a conclusion, we can infer that conclusion.

In `forward_chain()`:

if all(`p in self.facts` for `p in premises`):

checks that all premises are known, and:

`self.facts.add(conclusion)`

derives the conclusion.

The while loop repeats this process until no new facts are added, allowing derived facts to trigger further rules.